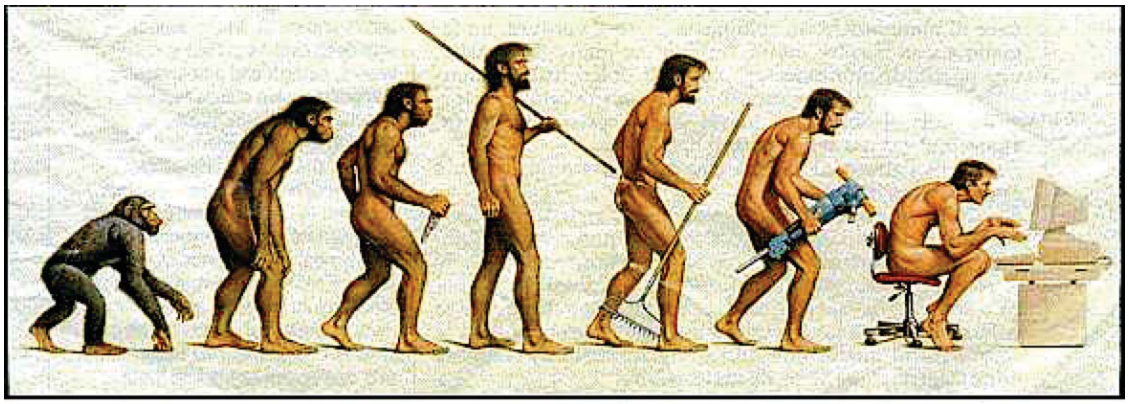
Computational Intelligence II: Evolutionary Optimisation

Chapter 1: Introduction



Somewhere, something went terribly wrong

Module provider: Dr. Baoru Huang

Course structure:

COMP 575 (15 credits) consist of two parts:

- Part I: Neural Networks
- Part II: Evolutionary Computation

Typically, about 4 hours every week:

- About 20 lectures for theory.
- About 12 tutorial sessions to show examples, demos/practical and Q&As.

Assessment:

- Final examination: MCQs

Recommended books for this part:

- Genetic Algorithms: in search optimisation and machine learning, DE Goldberg
- Genetic Algorithms + Data Structures = Evolution Programs, Z Michalewicz
- Genetic Programming: An Introduction, W Banzhaf, P Nordin, RE Keller, FD Francone
- An Introduction to Genetic Algorithms for Scientists and Engineers , DA Coley
- A Genetic Algorithm Tutorial, Darrell Whitley

Course contents:

⌘ **Introduction**
- ⦿ Chapter 1

⌘ **Genetic Algorithms**
- ⦿ Chapter 2 (basic elements)
- ⦿ Chapter 3 (advanced topics)
- ⦿ Chapter 4 (theoretical analysis)

⌘ **Genetic Programming**
- ⦿ Chapter 5 (overview & gene expression programming)

⌘ **Particle Swarm Optimisation**
- ⦿ Chapter 6 (overview and variations)

⌘ **Evolutionary Strategies**
- ⦿ Chapter 7 (overview and extensions)

Mathematical optimisation

- Optimisation is probably the most useful scientific topic due to its immense applicability in many scientific areas.

- What is **optimisation**?

- Optimisation is the process by which you can find the best design of a given system or model.

A set of specific parameter values that identifies a system.

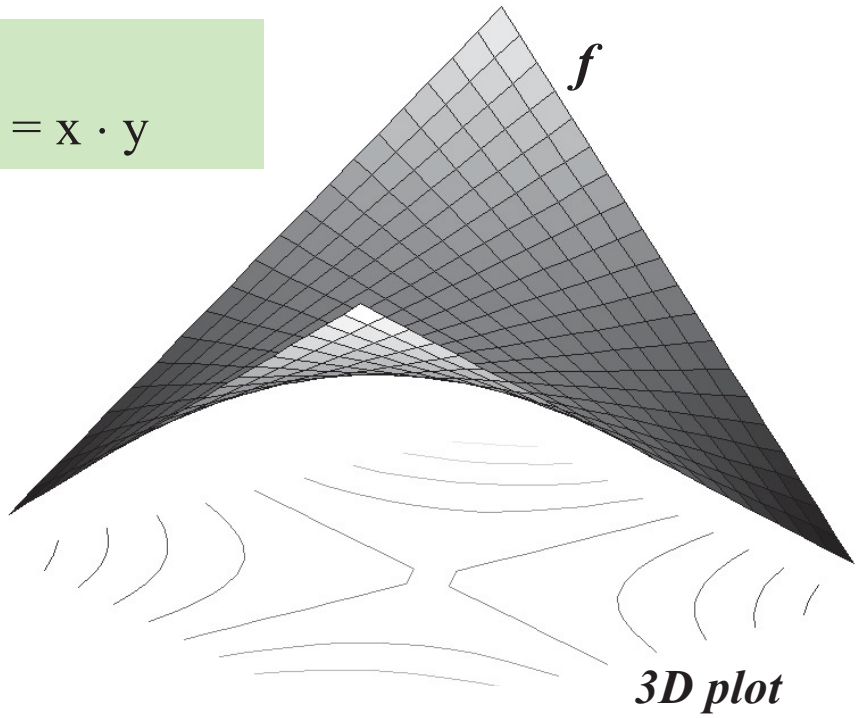
A set of measurable math formulae, goals, constraints and/or relationships that correspond to a problem at hand.

Which maximises some well-defined and measurable objective function.

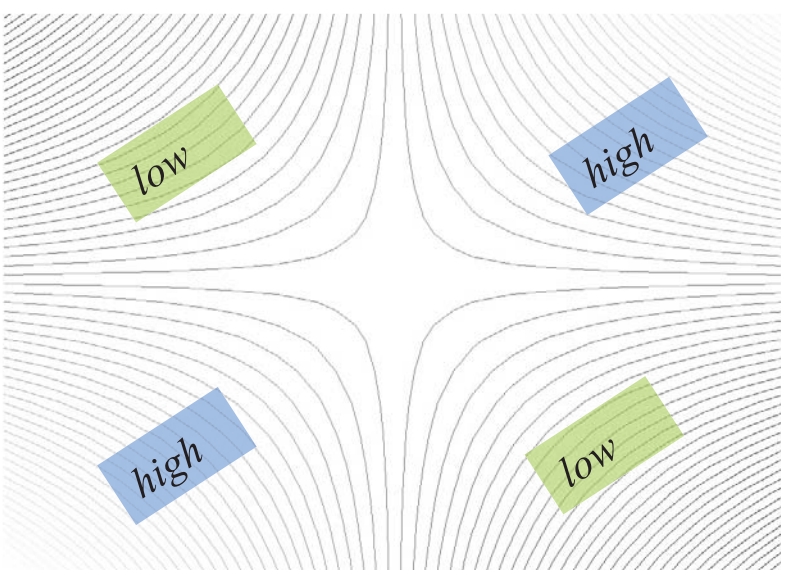
Example: unconstrained optimisation

Maximise:

$$f(x,y) = x \cdot y$$



contour plot



Example: constrained optimisation

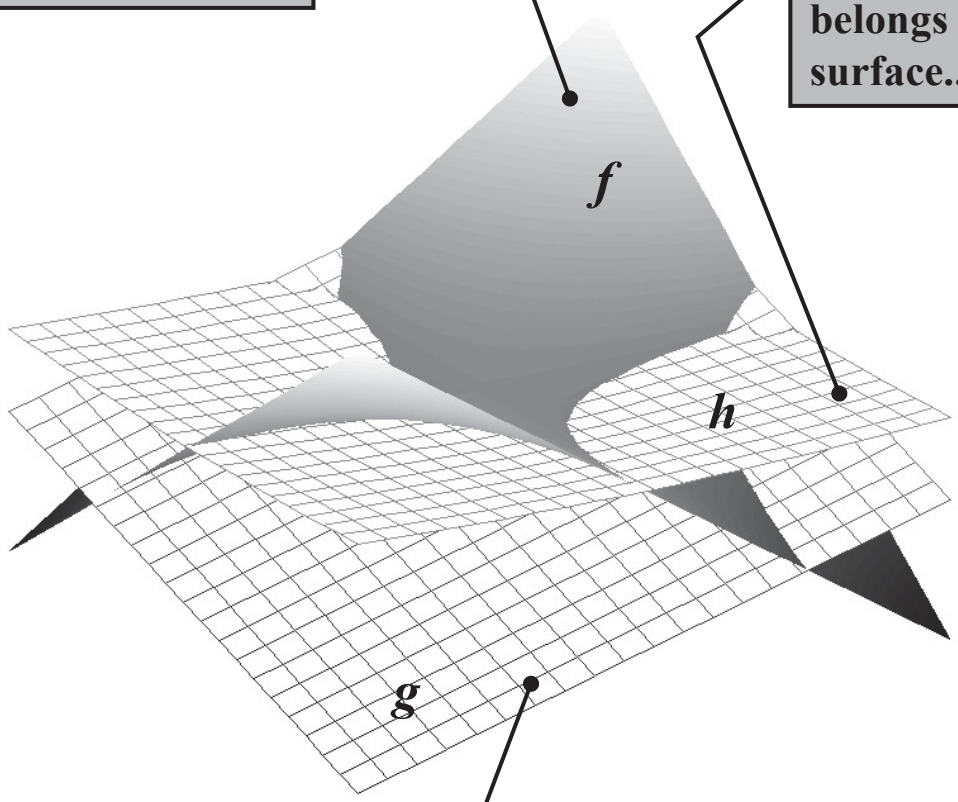
max $f(x,y) = x \cdot y$

s.t. : $g(x,y) = 2 \cdot (x + y) - 8.2 \leq 0$

$h(x,y) = 0.5 \cdot \sqrt{(x^2 + y^2)} - 1.8 = 0$

Choose the highest point on this surface...

...which also belongs to this surface...



… and, at the same time, does not lie above this surface.

Enumerative methods

- The simplest approach to global minimisation problems is simply to calculate all possible values of all of the parameters; this is called enumeration.

- Where parameters are continuous, it is necessary to create a discretised version of the allowable parameter values.

- This technique is simple, but very inefficient and impractical for large dimensions.

- Some more sophisticated versions of the basic enumeration method exist (e.g., Dynamic Programming) to improve the computational efficiency, but they still suffer from *the "curse of dimensionality"*.

Gradient-based methods

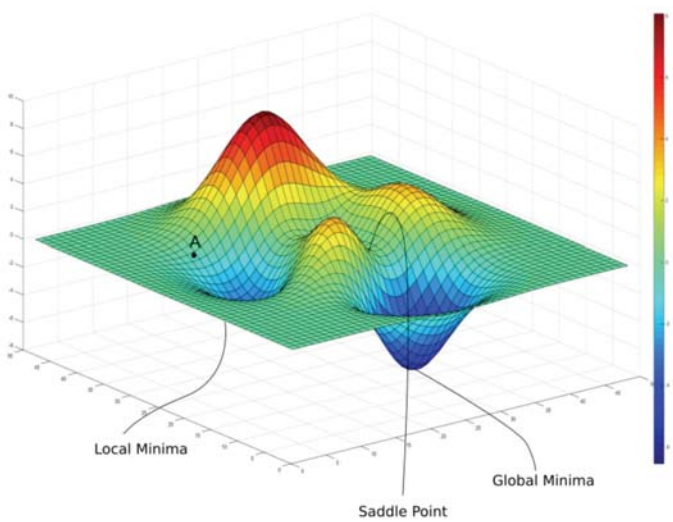
- Gradient-based methods operate by selecting a point x(t) and estimating the local gradient, and then selecting a new point x(t+1) in the direction of the steepest gradient (for maximisation) or largest negative gradient (for minimisation):

$$\min_{\vec{x}\in\Re^d} f\left(\vec{x}\right)$$

$$\vec{x}\left(t+1\right)=\vec{x}\left(t\right)-\eta\nabla f\left[\vec{x}\left(t\right)\right]$$

- This estimate of the gradient could be derived from differentiation of the equations (if possible!).

- These methods are good at finding local minima or maxima, but they only take the local properties of the cost function into account and will only find global extremum when they are initialised sufficiently near to the desired optimum solution.



Stochastic methods

- The biggest deficiency of the gradient-based methods is that their search strategies use only the local properties of the cost function. Most stochastic methods do not have this disadvantage since they choose new trial solutions according to rules that include randomised elements.

- That is not to say that each direction is equally likely, since most stochastic methods do bias the search toward a potentially optimum solution.

- Simple undirected (i.e. unbiased) algorithms such as the random walk do exist, but they are unlikely to be used for anything other than simple problems.

- Genetic Algorithms are a particular class of directed random search, but there are many other ones as well.

<END of Chapter 1>